

# Vision-Language-Action (VLA) Models

NECKER

August 30, 2026

## Contents

|          |                                                                                               |           |
|----------|-----------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Formal Definitions and Multi-Modal Vector Spaces</b>                                       | <b>2</b>  |
| 1.1      | The Visual Space (Observation) . . . . .                                                      | 2         |
| 1.2      | The Linguistic Space (Instruction) . . . . .                                                  | 2         |
| 1.3      | The Action Space (Proprioception and Actuation) . . . . .                                     | 2         |
| <b>2</b> | <b>Phase 1: Visual Ingestion and Modal Projection</b>                                         | <b>3</b>  |
| 2.1      | Patchification and Vision Transformer (ViT): From Pixel Grid to Semantic Vectors . . . . .    | 3         |
| 2.1.1    | A. The Quadratic Complexity Problem and the Patchification Solution . . . . .                 | 3         |
| 2.1.2    | B. Flattening and Linear Projection ( $W_{\text{patch}}$ ) . . . . .                          | 4         |
| 2.1.3    | C. Geometry Injection: Positional Embedding ( $\mathbf{E}_{\text{pos}}$ ) . . . . .           | 5         |
| 2.1.4    | D. Visual Deduction Engine: Self-Attention and $Q, K, V$ Matrix Dynamics in ViT . . . . .     | 6         |
| 2.2      | The Multi-Modal Adapter (Projector) . . . . .                                                 | 8         |
| 2.2.1    | Alignment and Projection (The Projector) . . . . .                                            | 8         |
| 2.2.2    | Multimodal Fusion: Ingestion by the Large Language Model . . . . .                            | 10        |
| <b>3</b> | <b>Phase 2: Physical Action Emission</b>                                                      | <b>11</b> |
| 3.1      | Paradigm 1: Autoregressive Action Generation . . . . .                                        | 11        |
| 3.1.1    | 1. Action Tokenization (Control Space Discretization): . . . . .                              | 11        |
| 3.1.2    | 2. Autoregressive Pose Decoding: . . . . .                                                    | 11        |
| 3.1.3    | 3. De-Quantization to Continuous Physical Domain: . . . . .                                   | 11        |
| 3.1.4    | 4. Low-Level Actuation: . . . . .                                                             | 12        |
| 3.2      | Paradigm 2: Continuous Generation via Conditional Flow Matching and Action Chunking . . . . . | 13        |
| 3.2.1    | 1. Two-Tier Architectural Topology . . . . .                                                  | 13        |
| 3.2.2    | 2. Mechanics of Action Chunking: Parallel Trajectory Prediction . . . . .                     | 14        |
| 3.2.3    | 5. Closed-Loop Execution: Receding Horizon and Temporal Ensembling . . . . .                  | 14        |
| 3.2.4    | 3. Mathematical Formulation: Conditional Flow Matching (CFM) . . . . .                        | 16        |
| 3.2.5    | 4. Runtime Inference and ODE Numerical Integration . . . . .                                  | 16        |
| 3.3      | On-Device Execution (Inference Time) . . . . .                                                | 19        |
| <b>4</b> | <b>Nature of the Visual Representation: Static Sensing vs. Dynamic Actuation</b>              | <b>19</b> |
| 4.1      | The Quasi-Markovian Problem Formulation . . . . .                                             | 19        |
| 4.1.1    | The Training Paradox: . . . . .                                                               | 19        |
| 4.2      | Physical Constraints of Single-Frame Perception . . . . .                                     | 19        |
| 4.3      | Architectural Compensation Strategies . . . . .                                               | 20        |
| 4.3.1    | a) Multi-Frame Sliding Window (Bounded Temporal Stacking): . . . . .                          | 20        |
| 4.3.2    | b) Historical Proprioception Conditioning: . . . . .                                          | 20        |
| 4.3.3    | c) Shifting Dynamics into Future Horizons (Action Chunking): . . . . .                        | 20        |

# 1 Formal Definitions and Multi-Modal Vector Spaces

Unlike pure Large Language Models (LLMs) that operate exclusively within the discrete space of textual language, **Vision-Language-Action (VLA)** models for robotics and Embodied AI (such as the architectures underlying OpenVLA or SmolVLA) are geometric reasoning engines that bridge continuous visual perception, discrete semantics, and physical actuation in the real world.

To formalize the architecture, we must define the three vector domains in which the model operates.

## 1.1 The Visual Space (Observation)

The primary input is a three-dimensional pixel matrix captured by sensors (e.g., an RGB camera mounted on the end-effector or within the workspace):

$$\mathcal{I} \in \mathbb{R}^{H \times W \times C}$$

where  $H$  and  $W$  denote spatial resolution (**height** and **width**) and  $C = 3$  represents the color channels. In the case of advanced manipulation (e.g., on a Franka Emika or a Dobot CR5), perception can extend to 3D point clouds, but the standard computational pipeline begins with RGB tensors.

## 1.2 The Linguistic Space (Instruction)

Human intent is provided as a sequence of textual tokens of length  $T_{\text{text}}$ , drawn from vocabulary  $\mathcal{V}$ :

$$\mathbf{L} = [l_1, l_2, \dots, l_{T_{\text{text}}}] \in \mathcal{V}^{T_{\text{text}}}$$

## 1.3 The Action Space (Proprioception and Actuation)

Unlike text, the output is not an abstract concept, but a kinematic state vector. We define action  $\mathbf{a}_t$  at time  $t$  as a continuous vector in the manipulator’s degree-of-freedom (DoF) space:

$$\mathbf{a}_t \in \mathbb{R}^{A_{\text{dim}}}$$

Typically,  $A_{\text{dim}} = 7$  (6 coordinates for the end-effector pose in Cartesian space  $\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}$  and 1 continuous scalar dimension for the gripper state).

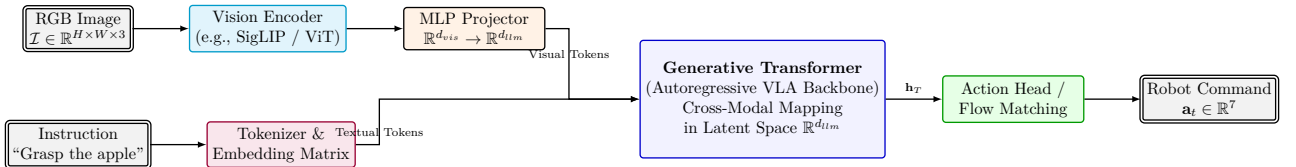


Figure 1: Macro-architecture of a VLA model. Visual and linguistic modalities collapse into the same Transformer latent space, which synthesizes context to produce physical control tensors.

## 2 Phase 1: Visual Ingestion and Modal Projection

The native language model possesses no innate concept of an "image". The image must be translated into the Transformer's "language": a sequence of dense vectors in  $\mathbb{R}^d$ .

### 2.1 Patchification and Vision Transformer (ViT): From Pixel Grid to Semantic Vectors

The task of the **Vision Encoder** is to translate the camera's continuous three-dimensional matrix into a discrete sequence of vectors.

#### 2.1.1 A. The Quadratic Complexity Problem and the Patchification Solution

Attempting to process an image pixel by pixel through a Transformer immediately encounters the mathematical barrier of *Self-Attention*. With its quadratic computational complexity  $O(N^2)$  relative to sequence length  $N$ , a standard resolution image of  $224 \times 224$  pixels would yield a sequence of  $N = 50,176$  pixels. Computing the attention matrix would require over 2.5 billion elementary operations per single layer, instantly saturating GPU VRAM.

To resolve this bottleneck, image  $\mathcal{I} \in \mathbb{R}^{H \times W \times C}$  is physically fragmented (partitioned) into a grid of non-overlapping square **patches** of fixed dimension  $P \times P$  (typically  $P = 14$  or  $P = 16$ ).

This operation, termed *Patchification*, drastically compresses sequence length. The total number of extracted patches becomes:

$$N_p = \frac{H \times W}{P^2}$$

For a  $224 \times 224$  image with  $14 \times 14$  patches, we obtain exactly  $N_p = 256$  patches. The image discards its global continuous structure and becomes a one-dimensional array of "tiles" from the original image.

**ATTENTION:** this is advantageous because the attention module no longer compares pixel by pixel as it would if each pixel were treated as an individual token; instead, it compares patch by patch, drastically slashing the required computations.

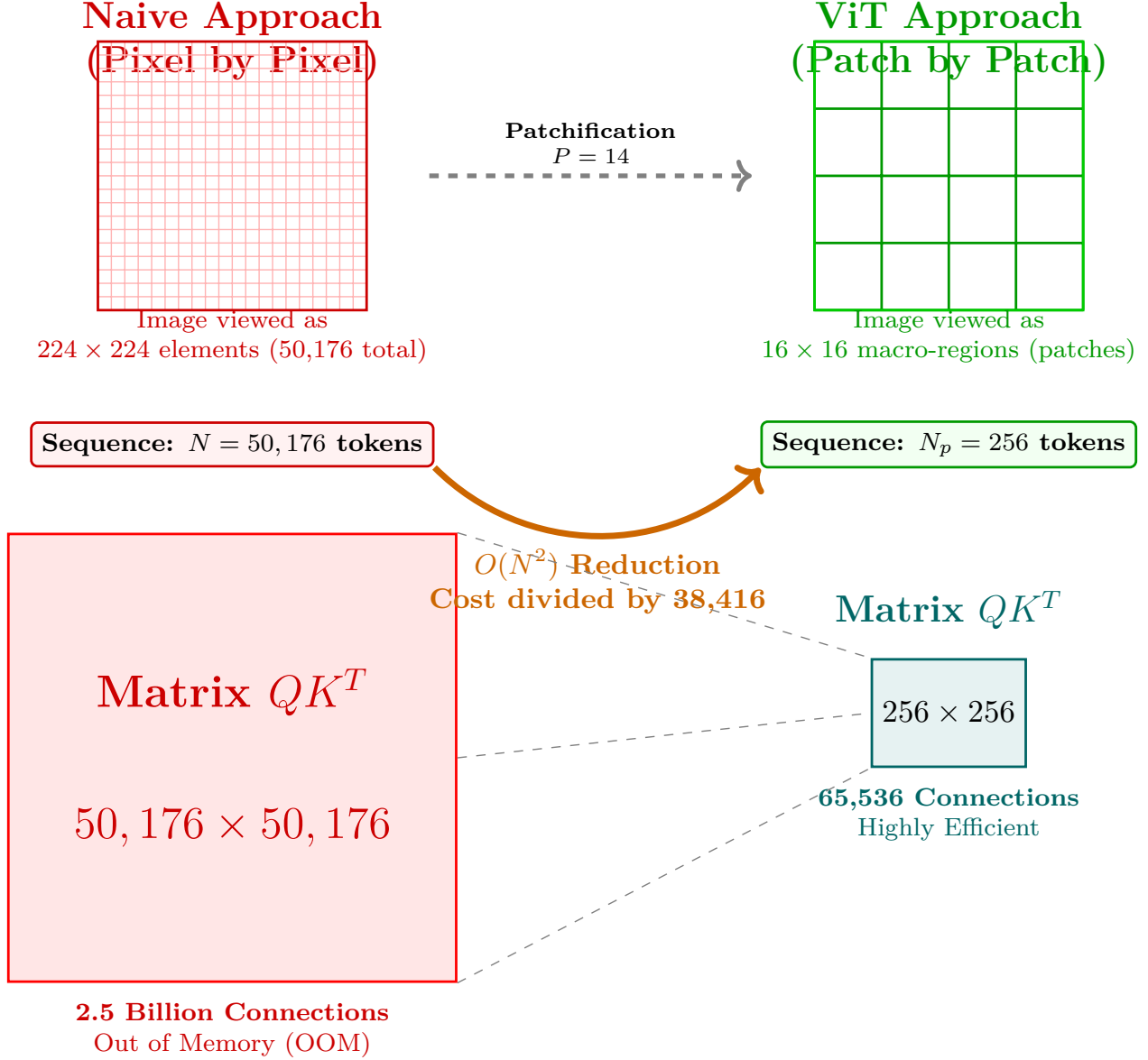


Figure 2: Geometric representation of computational complexity in *Self-Attention*. On the left, the dense approach (pixel-level) generates an  $N \times N$  affinity matrix physically intractable in RAM. On the right, *Patchification* compacts spatial data, drastically reducing attention matrix dimensions while preserving the model’s capacity to correlate distant image regions.

### 2.1.2 B. Flattening and Linear Projection ( $W_{\text{patch}}$ )

Each individually extracted patch remains a tiny 3D tensor of size  $P \times P \times C$  (e.g.,  $14 \times 14 \times 3$ ). Before processing, it must undergo **Flattening** (unrolling in memory), whereby pixel rows of **each patch** are unrolled sequentially into a single raw row vector of dimension  $P^2 \cdot C$  (e.g.,  $14 \times 14 \times 3 = 588$  decimal numbers, arranged for instance red first, then green, then blue).

However, these 588 numbers represent solely light intensity values (RGB), lacking true semantic depth. They are consequently multiplied by a static weight matrix  $W_{\text{patch}}$ , operating analogously to the embedding matrix  $W_E$  in language models:

$$\mathbf{x}_{\text{linear}}^{(i)} = \text{Flatten}(\text{Patch}_i) \cdot W_{\text{patch}} \in \mathbb{R}^{1 \times d_{\text{vis}}}$$

where  $W_{\text{patch}} \in \mathbb{R}^{(P^2 \cdot C) \times d_{\text{vis}}}$ . From an engineering perspective, this matrix multiplication is mathematically equivalent to a 2D convolution (Conv2D) with *kernel size* equal to  $P$  and *stride* equal to  $P$ . This operation projects the raw tile into a broader latent space (typically  $d_{\text{vis}} = 768$  or  $1152$ ), initiating the extraction of foundational geometric features (such as edges, corners, or color gradients).

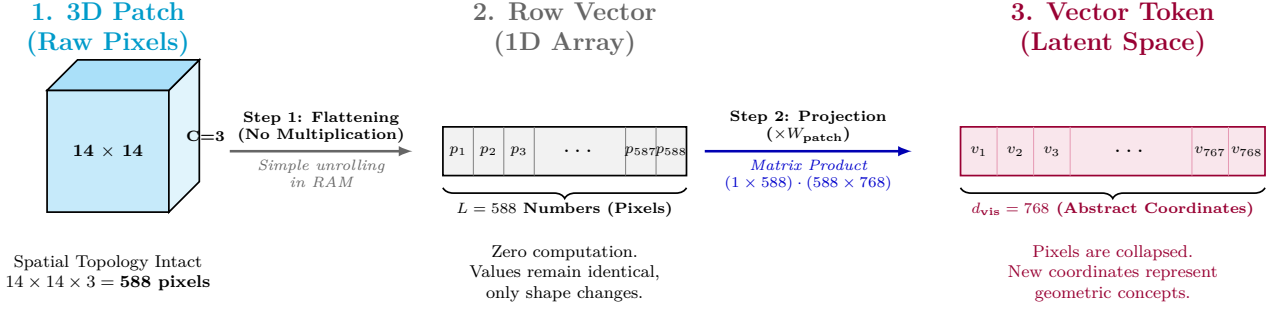


Figure 3: Decomposition of pre-processing operations on a single patch. Flattening is a structural data-resaping step. Projection ( $W_{\text{patch}}$ ) is the linear algebraic operation altering logical dimensionality, converting 588 visual intensities into 768 semantic coordinates.

### 2.1.3 C. Geometry Injection: Positional Embedding ( $\mathbf{E}_{\text{pos}}$ )

At this stage, we have 256 vectors; however, the Transformer architecture is intrinsically *permutation-invariant*: it possesses no notion of "up", "down", "right", or "left". Feeding these vectors unaltered would leave the robot incapable of distinguishing whether an object resides in the upper-left or lower-right region, precluding the computation of kinematic grasping coordinates.

This spatial amnesia is remedied by algebraically adding a **Positional Embedding** ( $\mathbf{E}_{\text{pos}}$ ) to each vector:

$$\mathbf{x}_p^{(i)} = \mathbf{x}_{\text{linear}}^{(i)} + \mathbf{E}_{\text{pos}}^{(i)}$$

The positional embedding  $\mathbf{E}_{\text{pos}}$  is a matrix of learnable parameters (or fixed sinusoidal functions) matching visual vector dimensionality. Adding the positional vector shifts coordinates within space  $d_{\text{vis}}$  so that each vector permanently "remembers" its originating location within the source image.

**Practical Example of Positional Injection:** To illustrate the algebraic effect of  $\mathbf{E}_{\text{pos}}$ , consider two identical objects (e.g., two manipulable cubes) located in opposing areas of the frame: the first in the top-left (Patch 1) and the second in the bottom-right (Patch 256).

Following only the linear projection  $W_{\text{patch}}$ , both vectors encode visual appearance alone (color, texture), rendering them numerically indistinguishable in a toy space  $d_{\text{vis}} = 4$ :

$$\mathbf{x}_{\text{linear}}^{(1)} = [0.90, 0.10, 0.05, 0.80], \quad \mathbf{x}_{\text{linear}}^{(256)} = [0.90, 0.10, 0.05, 0.80]$$

Assigning a unique positional vector  $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{d_{\text{vis}}}$  to each grid coordinate:

$$\mathbf{E}_{\text{pos}}^{(1)} = [+1.00, +1.00, 0.00, 0.00] \quad (\text{Top-Left})$$

$$\mathbf{E}_{\text{pos}}^{(256)} = [0.00, 0.00, -1.00, -1.00] \quad (\text{Bottom-Right})$$

Element-wise addition produces two distinct visual tokens in both semantics and spatial grounding:

$$\mathbf{x}_p^{(1)} = \mathbf{x}_{\text{linear}}^{(1)} + \mathbf{E}_{\text{pos}}^{(1)} = [1.90, 1.10, 0.05, 0.80]$$

$$\mathbf{x}_p^{(256)} = \mathbf{x}_{\text{linear}}^{(256)} + \mathbf{E}_{\text{pos}}^{(256)} = [0.90, 0.10, -0.95, -0.20]$$

Thanks to this latent-space shift, the *Self-Attention* module gains the metric coordinates necessary to plan kinematic trajectories toward the correct target.

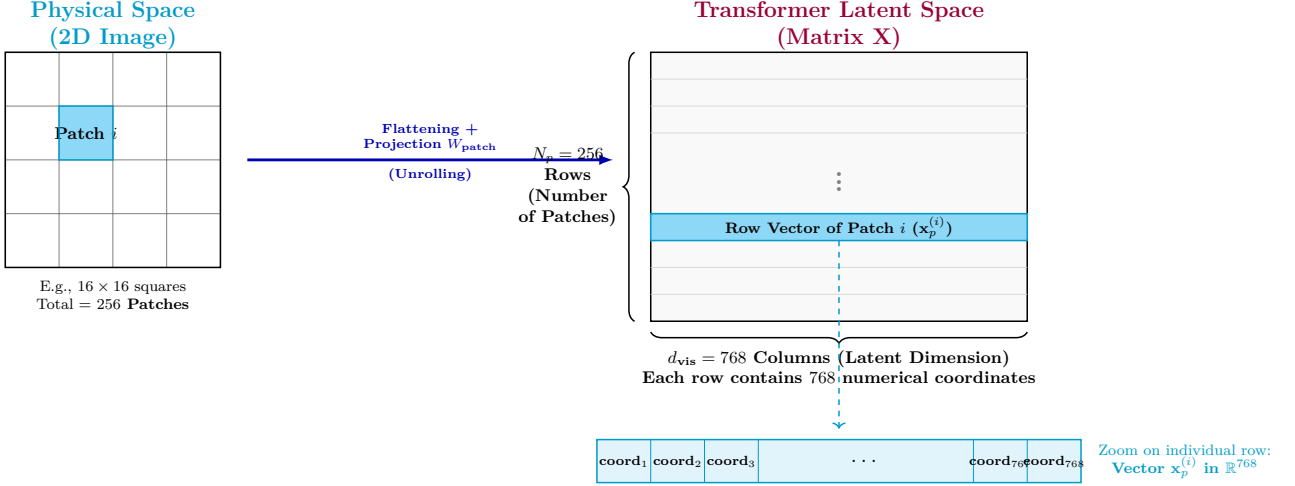


Figure 4: Geometric correspondence between pixel space and tensor space. Original image dimensions  $X$  and  $Y$  collapse into the vertical matrix axis (forming a sequence of 256 elements). The horizontal axis denotes semantic depth ( $d_{\text{vis}} = 768$ ), which is the number of mathematical coordinates used to describe the contents of that individual patch.

#### 2.1.4 D. Visual Deduction Engine: Self-Attention and $Q, K, V$ Matrix Dynamics in ViT

Vectors  $\mathbf{x}_p^{(i)} \in \mathbb{R}^{1 \times d_{\text{vis}}}$ , now endowed with initial photometric signatures and geometric constraints  $\mathbf{E}_{\text{pos}}$ , are stacked into input activation matrix  $\mathbf{Z}^{(0)} \in \mathbb{R}^{N_p \times d_{\text{vis}}}$ . This matrix enters a sequence of  $L_{\text{vis}}$  **ViT** blocks, driving semantic and relational abstraction.

Each visual attention block operates across three independent linear projections parametrized by weight matrices  $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{vis}} \times d_{\text{vis}}}$ :

$$Q = \mathbf{Z}^{(l-1)} W_Q, \quad K = \mathbf{Z}^{(l-1)} W_K, \quad V = \mathbf{Z}^{(l-1)} W_V$$

**1. Routing Role of  $Q$  and  $K$  (Affinity Geometry):** Matrices  $W_Q$  and  $W_K$  do not forward descriptive image features to subsequent layers; they uniquely establish *which patch attends to which*. For every patch pair  $i$  and  $j$ , the dot product between Query vector  $\mathbf{q}_i$  (patch  $i$ 's contextual query) and Key vector  $\mathbf{k}_j$  (patch  $j$ 's compatibility descriptor) evaluates topological and semantic affinity in latent space:

$$S_{i,j} = \frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}} \in \mathbb{R}$$

Applying the row-wise Softmax function maps raw scalar scores into a stochastic routing matrix  $A \in \mathbb{R}^{N_p \times N_p}$ , where each coefficient  $\alpha_{i,j} \in [0, 1]$  dictates the proportion of information patch  $i$  retrieves from patch  $j$ :

$$A = \text{Softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) \implies \sum_{j=1}^{N_p} \alpha_{i,j} = 1.0 \quad \forall i \in \{1, \dots, N_p\}$$

**2. Role of  $V$  (The Value Vector as Visual Payload Carrier):** Matrix  $W_V$  fulfills a structurally distinct role: it is the exclusive conduit for the **transformation and transport of actual perceptual content**.

Unlike  $Q$  and  $K$ , which collapse into scalar routing weights  $A$ , the row vectors of matrix  $V = [\mathbf{v}_1; \mathbf{v}_2; \dots; \mathbf{v}_{N_p}] \in \mathbb{R}^{N_p \times d_{\text{vis}}}$  represent transformed informational *payload*:

- Patch  $j$  is linearly projected via  $W_V$  into a new geometric subspace, isolating complex patterns (e.g., edge continuity, depth contrast, membership in a rigid surface, or grasping affordances for the end-effector).
- Vector  $\mathbf{v}_j = \mathbf{z}_j W_V$  represents the photometric and geometric distillate that patch  $j$  offers to the entire visual scene.

**3. Attention Synthesis and Residual Stream Update:** The visual attention module computes the product of routing weight matrix  $A$  and payload matrix  $V$ :

$$\text{Output}_{\text{Attn}} = A \cdot V \in \mathbb{R}^{N_p \times d_{\text{vis}}}$$

Examining the  $i$ -th row, patch  $i$ 's updated state is the convex linear combination of all Value vectors across the image, weighted by affinity:

$$\mathbf{z}_{\text{attn}}^{(i)} = \sum_{j=1}^{N_p} \alpha_{i,j} \mathbf{v}_j = \alpha_{i,1} \mathbf{v}_1 + \alpha_{i,2} \mathbf{v}_2 + \cdots + \alpha_{i,N_p} \mathbf{v}_{N_p}$$

- If patch  $i$  contains the metallic texture of a handle and patches  $j \in \mathcal{S}_{\text{door}}$  comprise the surrounding wooden panel,  $QK^T$  assigns elevated weights  $\alpha_{i,j}$  for  $j \in \mathcal{S}_{\text{door}}$ .
- In the weighted sum, updated vector  $\mathbf{z}_{\text{attn}}^{(i)}$  absorbs substantial portions of the door's vectors  $\mathbf{v}_j$ , embedding global door properties into the handle's local representation.

This aggregated state passes through output projection matrix  $W_O \in \mathbb{R}^{d_{\text{vis}} \times d_{\text{vis}}}$  and is merged into the flow via the first residual connection:

$$\mathbf{z}_{\text{mid}}^{(i)} = \mathbf{z}_{\text{input}}^{(i)} + \mathbf{z}_{\text{attn}}^{(i)} W_O$$

**ATTENTION:**  $\mathbf{z}_{\text{attn}}$  is summed with  $\mathbf{z}_{\text{input}}$  to preserve original local photometric features, preventing global contextual averaging from washing them out.

**4. RMSNorm Stabilization and MLP Module (Single-Patch Deduction):** Intermediate vector  $\mathbf{z}_{\text{mid}}^{(i)}$  completes horizontal cross-patch communication, yet does not constitute the final block output. The features must now be processed by the second sub-module: the **Feed-Forward Network (MLP)**.

While Self-Attention operates *cross-patch*, the MLP operates on each patch in **strict positional isolation** (*point-wise*):

1. **Intermediate Normalization:** Vector  $\mathbf{z}_{\text{mid}}^{(i)}$  traverses a second normalization stage to stabilize variance prior to expansion:

$$\mathbf{a}_{\text{mlp}}^{(i)} = \text{RMSNorm}(\mathbf{z}_{\text{mid}}^{(i)}) \in \mathbb{R}^{1 \times d_{\text{vis}}}$$

2. **Non-Linear Expansion and Gating (SwiGLU / GeLU):** The vector is projected into an expanded intermediate dimension  $d_{\text{ff}}$  (e.g.,  $d_{\text{ff}} \approx 4d_{\text{vis}}$ ) where non-linear activations process complex conceptual patterns before recompressing to  $d_{\text{vis}}$ :

$$\text{MLP}(\mathbf{a}_{\text{mlp}}^{(i)}) = \left( \text{Swish}(\mathbf{a}_{\text{mlp}}^{(i)} W_{\text{gate}}) \odot (\mathbf{a}_{\text{mlp}}^{(i)} W_{\text{up}}) \right) W_{\text{down}}$$

3. **Second Residual Connection and Layer Output:** The MLP output is added algebraically to  $\mathbf{z}_{\text{mid}}^{(i)}$ , concluding the  $l$ -th layer:

$$\mathbf{z}_{\text{out}}^{(i)} = \mathbf{z}_i^{(l)} = \mathbf{z}_{\text{mid}}^{(i)} + \text{MLP}(\mathbf{a}_{\text{mlp}}^{(i)})$$

**5. Parallel Propagation to Subsequent Blocks ( $l \rightarrow l+1$ ):** All  $N_p = 256$  patches complete this sequence, yielding the updated matrix:

$$\mathbf{Z}^{(l)} = \begin{bmatrix} \mathbf{z}_1^{(l)} \\ \mathbf{z}_2^{(l)} \\ \vdots \\ \mathbf{z}_{N_p}^{(l)} \end{bmatrix} \in \mathbb{R}^{N_p \times d_{\text{vis}}}$$

This becomes the direct input  $\mathbf{Z}_{\text{input}}^{(l+1)}$  for the next layer (each layer extracting progressively more abstract features). This sequence cascades across all  $L_{\text{vis}}$  Vision Transformer blocks until the final representation  $\mathbf{V}_{\text{raw}} = \mathbf{Z}^{(L_{\text{vis}})}$  is extracted.

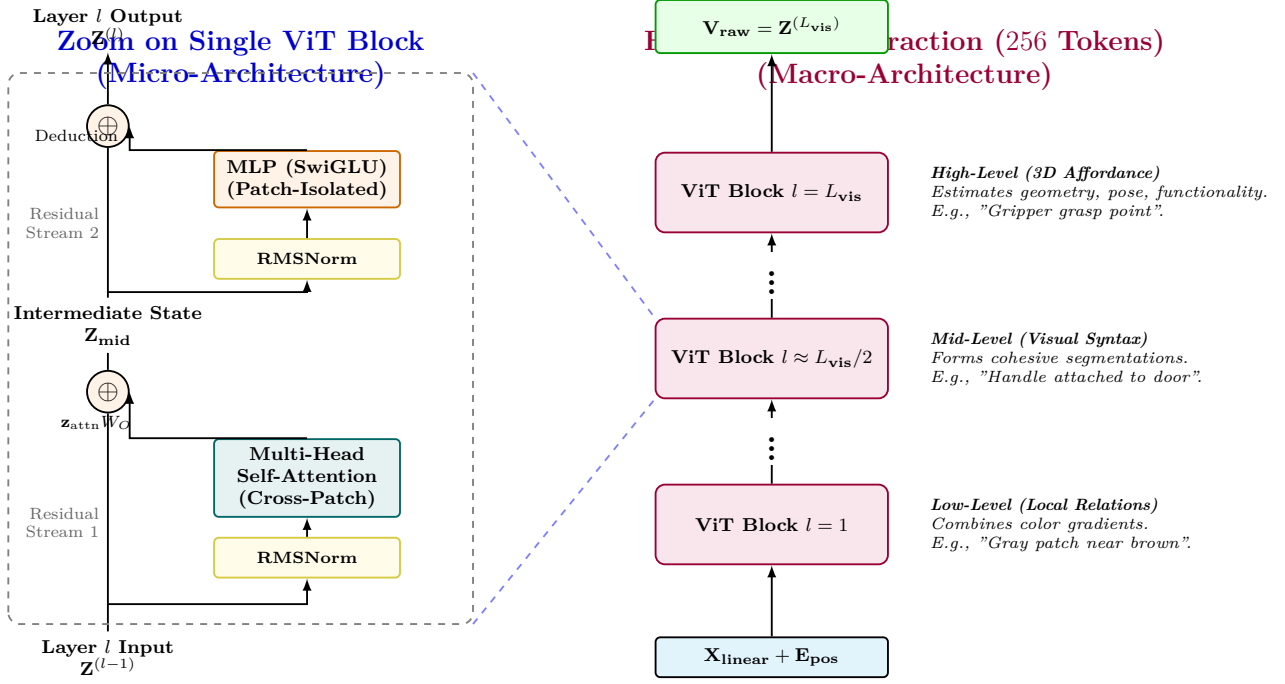


Figure 5: Overview of the visual reasoning engine. Left: internal architecture of single block  $l$  (highlighting data preservation via  $\mathbf{Z}_{\text{mid}}$  and the complementary roles of Attention and MLP). Right: cyclic repetition across full depth ( $L_{\text{vis}}$ ). At each transition ( $l \rightarrow l + 1$ ), layer math maps tensors from *raw photometric features* into *global semantic and kinematic structures* ready for action decoding.

## 2.2 The Multi-Modal Adapter (Projector)

At the conclusion of *Vision Encoder* processing, the resulting matrix  $\mathbf{V}_{\text{raw}} \in \mathbb{R}^{N_p \times d_{\text{vis}}}$  contains the operational scene semantics. However, these vectors cannot enter the Large Language Model directly due to dimensional and semantic mismatches:

- **Dimensional Mismatch:** The Vision Encoder and LLM feature different latent dimensions (e.g.,  $d_{\text{vis}} = 1152$  versus  $d_{\text{llm}} = 4096$ ).
- **Semantic Mismatch:** Coordinates in  $\mathbf{V}_{\text{raw}}$  encode visual topology, whereas the LLM structures its latent space around linguistic and grammatical relationships.

### 2.2.1 Alignment and Projection (The Projector)

To bridge this gap, the visual matrix passes through an adapter named the **Projector** (commonly a two-layer MLP). Its mathematical function is to project visual tokens into the language embedding space.

Geometrically, the Projector performs rigid rotation, translation, and dimensional expansion. Functionally, it translates: taking a vector geometrically representing a "*horizontal red cylinder*" and adjusting its coordinates so the LLM reads it identically to the text tokens corresponding to those words.

**Mathematical Anatomy of the Multimodal Projector:** The multimodal projector (found in architectures like LLaVA, OpenVLA, and Prismatic VLMs) acts as a continuous non-linear map between two distinct Riemannian manifolds: the visual feature space  $\mathcal{M}_{\text{vis}} \subset \mathbb{R}^{d_{\text{vis}}}$  and the language concept space  $\mathcal{M}_{\text{llm}} \subset \mathbb{R}^{d_{\text{llm}}}$ .

The standard configuration is a **Two-Layer Multilayer Perceptron (MLP)** with expanded intermediate projection and non-linear activation:

$$\mathbf{v}_{\text{llm}}^{(i)} = f_{\text{proj}} \left( \mathbf{v}_{\text{vis}}^{(i)} \right) = \left( \sigma \left( \mathbf{v}_{\text{vis}}^{(i)} W_1 + \mathbf{b}_1 \right) \right) W_2 + \mathbf{b}_2$$

where tensor dimensions are defined as:

- $\mathbf{v}_{\text{vis}}^{(i)} \in \mathbb{R}^{1 \times d_{\text{vis}}}$ : single visual token output by the final ViT block (e.g.,  $d_{\text{vis}} = 1152$  for SigLIP-SO400M).

- $W_1 \in \mathbb{R}^{d_{\text{vis}} \times d_{\text{llm}}}$  and  $\mathbf{b}_1 \in \mathbb{R}^{1 \times d_{\text{llm}}}$ : first affine transformation matrix aligning dimensions across both domains.
- $\sigma(\cdot) = \text{GELU}(\cdot)$ : *Gaussian Error Linear Unit* activation function. Non-linearity is critical: it folds and shapes the visual vector space, ensuring the correspondence between perceptual features and text semantics is not constrained to an isometric linear map. (GELU is selected because it avoids dying neurons in compact, non-redundant projection networks.)
- $W_2 \in \mathbb{R}^{d_{\text{llm}} \times d_{\text{llm}}}$  and  $\mathbf{b}_2 \in \mathbb{R}^{1 \times d_{\text{llm}}}$ : final linear refinement layer aligning the latent distribution with the LLM’s embedding weights (e.g.,  $d_{\text{llm}} = 4096$  for Llama-2-7B or Qwen-2.5-7B).

This operation applies row-wise in parallel across all tokens in  $\mathbf{V}_{\text{raw}}$ , preserving token count  $N_p$ :

$$\mathbf{V}_{\text{llm}} = f_{\text{proj}}(\mathbf{V}_{\text{raw}}) \in \mathbb{R}^{N_p \times d_{\text{llm}}}$$

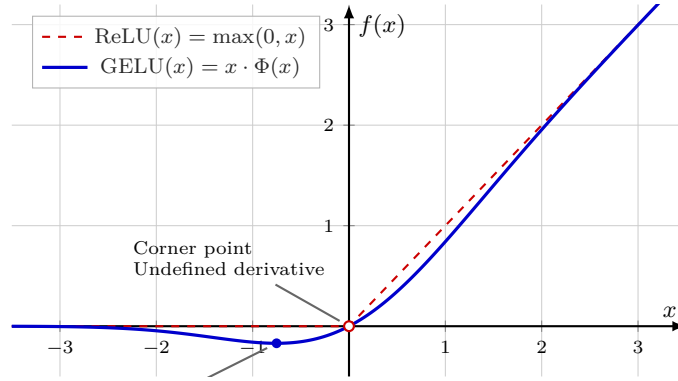


Figure 6: Functional comparison between  $\text{GELU}(x)$  and  $\text{ReLU}(x)$ . Unlike the non-differentiable singularity of  $\text{ReLU}$  at  $x = 0$ ,  $\text{GELU}$  is continuously differentiable throughout its domain ( $C^\infty$ ) and exhibits a strictly negative region for  $x \in (-3, 0)$ , mitigating dying neurons and ensuring stable convergence during multimodal alignment in the Projector.

**Intuitive Example of Semantic Translation:** To illustrate this transition algebraically, consider a reduced-dimension toy setting ( $d_{\text{vis}} = 3$ ,  $d_{\text{llm}} = 5$ ) applied to a patch depicting a red ceramic mug:

1. **In Visual Space ( $\mathbb{R}^3$ ):** The Vision Encoder captures only local photometric intensity and reflectance:

$$\mathbf{v}_{\text{vis}} = [ \underbrace{0.92}_{\text{Red chrominance}}, \underbrace{0.10}_{\text{Curvature gradient}}, \underbrace{0.85}_{\text{Specular reflection}} ]$$

2. **Transformation via  $f_{\text{proj}}$ :** Through multiplication by  $W_1$ , GELU activation, and alignment matrix  $W_2$ , the three photometric inputs are combined and mapped across 5 latent dimensions.

3. **In Linguistic Space ( $\mathbb{R}^5$ ):** The LLM structures axes around abstract properties and causal relationships:

$$\mathbf{v}_{\text{llm}} = [ \underbrace{0.78}_{\text{Rigid object}}, \underbrace{0.02}_{\text{Animate}}, \underbrace{0.91}_{\text{Grasp/Affordance}}, \underbrace{0.63}_{\text{Container}}, \underbrace{0.69}_{\text{Warm tone}} ]$$

The resulting vector  $\mathbf{v}_{\text{llm}}$  is no longer a raw optical descriptor: it resides on the manifold of the LLM’s text tokens, allowing the model to interpret it just like the syntactic phrase “*graspable red mug*”, enabling subsequent trajectory synthesis.

**ATTENTION:** this translation is not literal text generation; rather, vectors representing brightness patterns or visual features map into vectors that interact directly with instructions, describing functional properties, affordances, manipulable entities, and causal relations.

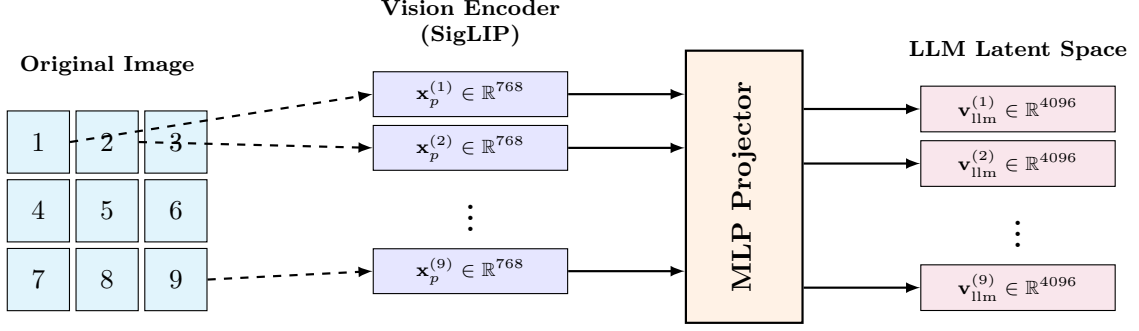


Figure 7: Mechanics of Patchification and Projection. The image sheds its two-dimensional topology and becomes a sequential matrix of vectors compatible with the autoregressive paradigm.

### 2.2.2 Multimodal Fusion: Ingestion by the Large Language Model

In parallel with visual acquisition, the robot receives a natural language instruction from the user (e.g., *"Grasp the red block and place it on the right"*). This phrase is processed by the LLM tokenizer and *Word Embedding layer*, generating textual token embeddings:

$$\mathbf{T}_{\text{emb}} = [\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_K] \in \mathbb{R}^{K \times d_{\text{llm}}}$$

where  $K$  is the number of subwords in the instruction.

Having aligned dimensionality ( $d_{\text{llm}}$ ), the VLA performs **Multimodal Concatenation**. Visual and textual embedding vectors are concatenated along the sequence axis into a unified input matrix for the LLM's Transformer blocks:

$$\mathbf{S}_{\text{input}} = [\mathbf{V}_{\text{llm}} \parallel \mathbf{T}_{\text{emb}}] \in \mathbb{R}^{(N_p + K) \times d_{\text{llm}}}$$

This hybrid sequence forms the foundation of Embodied AI:

- The initial  $N_p$  embeddings deliver the scene's visual and geometric context to the LLM.
- The subsequent  $K$  embeddings deliver the intentional constraint (the objective).

Matrix  $\mathbf{S}_{\text{input}}$  is processed through the LLM's causal layers, where Self-Attention enables linguistic tokens to attend to visual *affordances*, culminating in the autoregressive generation of **Action Tokens**.

### 3 Phase 2: Physical Action Emission

A standard LLM uses its final hidden vector  $\mathbf{h}_T$  to query the unembedding matrix  $W_U$  and predict a subsequent word token. In Embodied AI, we must generate physical actuation. Two major mathematical paradigms address this challenge.

#### 3.1 Paradigm 1: Autoregressive Action Generation

The concatenated matrix  $\mathbf{S}_{\text{input}} \in \mathbb{R}^{(N_p+K) \times d_{\text{llm}}}$  enters the stack of  $L_{\text{llm}}$  Decoder blocks in the Large Language Model (e.g., Llama or Qwen).

At this stage, Self-Attention operates in a hybrid fashion:

- Visual tokens  $\mathbf{V}_{\text{llm}}$  can interact bidirectionally (or through a causal mask depending on the specific VLM setup).
- Textual tokens  $\mathbf{T}_{\text{emb}}$  query visual tokens: each subword of the instruction (e.g., *"block"*, *"grasp"*) computes dot products  $QK^T$  against visual patches, isolating target coordinates and grasp affordances in workspace coordinates.

##### 3.1.1 1. Action Tokenization (Control Space Discretization):

A standard LLM predicts over a discrete vocabulary  $\mathcal{V}$  via a linear classification head (*LM Head*) followed by a Softmax. To control a serial manipulator (typically with 7 degrees of freedom:  $\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}$ , gripper), frameworks such as RT-2 and OpenVLA use **Action Tokenization**:

1. **Uniform Binned Discretization:** Each continuous action dimension  $a_d \in [a_{\min}, a_{\max}]$  is uniformly quantized into  $B$  discrete bins (typically  $B = 256$ ):

$$\text{bin}(a_d) = \left\lfloor \frac{a_d - a_{\min}}{a_{\max} - a_{\min}} \cdot (B - 1) \right\rfloor \in \{0, 1, \dots, B - 1\}$$

2. **Dedicated Vocabulary / Token Overwriting:** These 256 bins map to 256 designated tokens within the LLM vocabulary (often repurposing unused text tokens or appending dedicated special tokens to the end of  $\mathcal{V}$ ).

##### 3.1.2 2. Autoregressive Pose Decoding:

Following the input sequence, the LLM autoregressively predicts the 7 consecutive action tokens via sampling or greedy decoding (argmax):

$$\mathbf{a}_{\text{discrete}} = [\tau_{\Delta x}, \tau_{\Delta y}, \tau_{\Delta z}, \tau_{\Delta \phi}, \tau_{\Delta \theta}, \tau_{\Delta \psi}, \tau_{\text{grip}}]$$

where the probability of each action token is conditioned on multimodal context as well as preceding action tokens:

$$P(\tau_k \mid \mathbf{S}_{\text{input}}, \tau_{<k}) = \text{Softmax}(\mathbf{z}_{\text{last}} W_{\text{LM}})_{\tau_k}$$

##### 3.1.3 3. De-Quantization to Continuous Physical Domain:

The predicted discrete tokens are analytically inverted to reconstruct continuous operational space coordinates:

$$\hat{a}_d = a_{\min} + \frac{\tau_d}{B - 1} \cdot (a_{\max} - a_{\min}) \in \mathbb{R}$$

The resulting continuous vector:

$$\mathbf{u}_t = [\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{grip}] \in \mathbb{R}^7$$

represents an incremental relative displacement (or target absolute Tool Center Point pose) in the robot's Cartesian reference frame.

### 3.1.4 4. Low-Level Actuation:

Vector  $\mathbf{u}_t$  is transmitted via middleware (e.g., ROS 2) to the robot’s motion planner:

- Inverse kinematics computes target joint configurations:  $\mathbf{q}^* = \text{IK}(\mathbf{u}_t)$ .
- Actuators (brushless motors / servos) execute motion via high-frequency closed-loop controllers (torque/velocity PD/PID), completing the perception-action cycle.

**End-to-End Numerical Example: Grasping a Cube** To illustrate the computational pathway from multimodal input to physical execution, consider a discrete control step at 10 Hz where a manipulator performs an incremental motion toward a target cube on a table.

1. **Multimodal Causal Conditioning:** The sequence  $\mathbf{S}_{\text{input}} \in \mathbb{R}^{(256+6) \times 4096}$  passes through the LLM. Attention heads link the word token “*grasp*” to visual patches corresponding to the target’s Cartesian location. The final hidden vector,  $\mathbf{z}_{\text{last}} \in \mathbb{R}^{1 \times 4096}$ , aggregates this joint kinematic intent.
2. **Autoregressive Action Token Emission:** The model emits a 7-DoF command in operational space:

$$\mathbf{u} = [\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{grip}]$$

Assuming translational displacements are bounded within  $[-0.10 \text{ m}, +0.10 \text{ m}]$  and discretized into  $B = 256$  bins (0 to 255), the LLM decodes:

$$\begin{aligned} \tau_{\Delta x} &= 140 \quad (\text{slight forward displacement along } X) \\ \tau_{\Delta y} &= 128 \quad (\text{neutral midpoint: zero lateral motion on } Y) \\ \tau_{\Delta z} &= 85 \quad (\text{downward descent along } Z) \\ \tau_{\Delta \phi}, \tau_{\Delta \theta}, \tau_{\Delta \psi} &= [128, 128, 128] \quad (\text{constant end-effector orientation}) \\ \tau_{\text{grip}} &= 0 \quad (\text{binary state: gripper open during approach}) \end{aligned}$$

3. **Continuous De-Quantization:** Each integer token  $\tau_d$  is converted back to metric units via inverted uniform quantization (this formula scales the decoded bin index to the actual operational range of the robot’s joints):

$$\begin{aligned} \Delta x &= -0.10 + \frac{140}{255} \cdot (0.10 - (-0.10)) = -0.10 + \frac{140}{255} \cdot 0.20 \approx +0.0098 \text{ m} \quad (+9.8 \text{ mm}) \\ \Delta y &= -0.10 + \frac{128}{255} \cdot 0.20 \approx +0.0004 \text{ m} \approx 0.0 \text{ mm} \\ \Delta z &= -0.10 + \frac{85}{255} \cdot 0.20 \approx -0.0333 \text{ m} \quad (-33.3 \text{ mm}) \end{aligned}$$

The resulting displacement vector for the Tool Center Point (TCP) is:

$$\mathbf{u}_t = [+0.0098 \text{ m}, 0.0 \text{ m}, -0.0333 \text{ m}, 0.0 \text{ rad}, 0.0 \text{ rad}, 0.0 \text{ rad}, 0.0]$$

4. **Inverse Kinematics and Motor Control:** The arm controller (e.g., a ROS 2 trajectory node) receives Cartesian delta  $\mathbf{u}_t$  at 10 Hz. Using the geometric Jacobian  $J(\mathbf{q}) \in \mathbb{R}^{6 \times n}$ , differential inverse kinematics yields:

$$\Delta \mathbf{q} = J^\dagger(\mathbf{q}) \cdot \mathbf{u}_{t(1:6)}$$

computing joint increments  $\Delta \mathbf{q} = [\Delta q_1, \dots, \Delta q_n]$  fed to actuator PID loops, moving the gripper toward the cube.

## ACTION HEAD ARCHITECTURE:

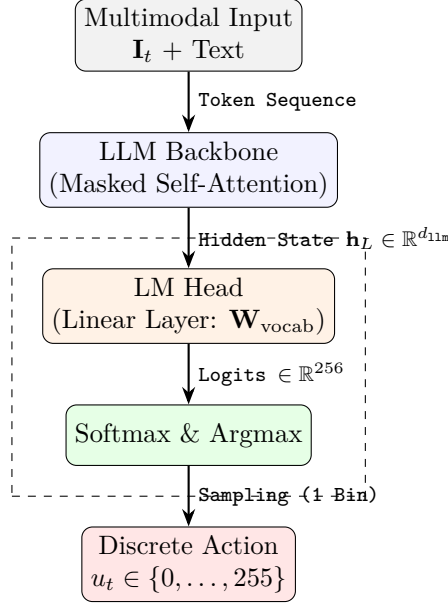


Figure 8: Discrete token Action Head architecture (Paradigm 1). The head is a single linear layer without temporal state tracking.

### 3.2 Paradigm 2: Continuous Generation via Conditional Flow Matching and Action Chunking

The limitations of discretization (quantization error on fixed grids, slow autoregressive decoding, and discontinuous setpoints sent to motors) motivated an architectural shift in second-generation VLAs (such as  $\pi_0$  and policies derived from ACT/Diffusion Policy).

In this framework, the model does not predict categorical distributions over discrete indices, but solves a **vector field regression** problem over continuous manifolds, uncoupling slow semantic reasoning from high-frequency motor control.

#### 3.2.1 1. Two-Tier Architectural Topology

Rather than forcing the autoregressive backbone to generate floating-point values directly, the system splits functional roles across two submodules:

1. **High-Level Semantic Backbone (Multimodal VLM):** Consists of the Vision Encoder and pre-trained Transformer layers (e.g., derived from LLaVA or PaliGemma). It ingests visual tokens  $\mathbf{V}_{\text{raw}}$  and instruction  $\mathbf{T}_{\text{emb}}$ , computing:

$$\mathbf{S}_{\text{input}} = [\mathbf{V}_{\text{llm}} \parallel \mathbf{T}_{\text{emb}}] \in \mathbb{R}^{(N_p+K) \times d_{\text{llm}}}$$

Instead of autoregressively generating text via an *LM Head*, the final layer produces dense latent activations. Using attention pooling or end-of-sequence token extraction, the backbone yields a contextual conditioning vector:

$$\mathbf{c} \in \mathbb{R}^{d_{\text{cond}}} \quad (\text{or sequence } \mathbf{C} \in \mathbb{R}^{M \times d_{\text{cond}}})$$

This vector summarizes high-level task intent (*"what to grasp, which obstacles to avoid, target location"*). Operating on abstract semantics, this module can execute at lower frequencies (e.g., 5–10 Hz).

2. **Low-Level Generative Action Head (Denoising / Vector Field Network):** A compact, low-latency module (typically a 4–8 layer bidirectional Transformer Decoder or ResNet MLP) parametrizing vector field:

$$\mathbf{v}_{\theta}(\mathbf{A}_s, s, \mathbf{c})$$

The Action Head takes as input:

- Current kinematic state  $\mathbf{A}_s \in \mathbb{R}^{H \times D}$  at flow step  $s$ .

- Continuous scalar time step  $s \in [0, 1]$ , encoded using Fourier embeddings (*Sinusoidal Positional Embeddings*):  $\mathbf{e}_s \in \mathbb{R}^{d_{\text{time}}}$ .
- Conditioning vector  $\mathbf{c}$ , incorporated via *Cross-Attention* or *AdaLN* (*Adaptive Layer Normalization*).
- Real-time proprioception  $\mathbf{q}_t \in \mathbb{R}^D$  (encoder angles or current TCP pose).

### 3.2.2 2. Mechanics of Action Chunking: Parallel Trajectory Prediction

Unlike step-by-step causal decoding, actions are produced in parallel:  $H$  future steps are synthesized simultaneously.

**Trajectory Tensor Definition:** The Action Head outputs a dense matrix:

$$\mathbf{A} = \begin{bmatrix} \mathbf{u}_t \\ \mathbf{u}_{t+1} \\ \vdots \\ \mathbf{u}_{t+H-1} \end{bmatrix} = \begin{bmatrix} a_{t,1} & a_{t,2} & \cdots & a_{t,D} \\ a_{t+1,1} & a_{t+1,2} & \cdots & a_{t+1,D} \\ \vdots & \vdots & \ddots & \vdots \\ a_{t+H-1,1} & a_{t+H-1,2} & \cdots & a_{t+H-1,D} \end{bmatrix} \in \mathbb{R}^{H \times D}$$

where  $H$  is the planning horizon (e.g.,  $H = 50$  steps) and  $D$  denotes actuated degrees of freedom ( $D = 7$ :  $\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}, \text{gripper}$ ).

**Temporal Correlation Management:** Inside the Action Head, horizon  $H$  is not decoded autoregressively, mitigating compounding errors:

- Initial noise matrix  $\mathbf{A}_0 \in \mathbb{R}^{H \times D}$  is treated as a sequence of  $H$  continuous tokens.
- Each row  $h \in \{0, \dots, H-1\}$  receives a learned 1D temporal positional embedding  $\mathbf{p}_h \in \mathbb{R}^{d_{\text{head}}}$ , indicating relative time offset ( $t+h$ ).
- **Bidirectional Self-Attention (non-causal)** allows all predicted future time steps to attend to one another: step  $t+10$  informs step  $t+1$ , enforcing smooth velocity and acceleration profiles across the trajectory chunk.

### 3.2.3 5. Closed-Loop Execution: Receding Horizon and Temporal Ensembling

Blindly executing the entire trajectory  $\mathbf{A}^*$  for all  $H$  steps without replanning would yield open-loop control for up to a second, leaving the manipulator prone to tracking drift or unpredicted collisions.

To preserve closed-loop responsiveness, deployment combines two methods:

**Receding Horizon Control (RHC):** The generative head is queried at frequency  $f_{\text{replan}} = \frac{1}{\Delta t \cdot K}$  (e.g., every  $K = 8$  steps, where  $K \ll H$ ). Only the first  $K$  actions of the predicted chunk are executed; at step  $t+K$ , the model processes updated camera frames, extracts a refreshed context vector  $\mathbf{c}$ , and replans the future horizon  $\mathbf{A}_{t+K:t+K+H}$ .

**Temporal Ensembling (Exponential Plan Smoothing):** Because consecutive plans generated at different steps may exhibit minor kinematic differences, the actual motor command  $\bar{\mathbf{u}}_t$  applied at time  $t$  is computed as a weighted convex combination of all active overlapping predictions:

$$\bar{\mathbf{u}}_t = \frac{\sum_{i=0}^{\min(t, H-1)} w_i \cdot \mathbf{A}_{t-i}[i]}{\sum_{i=0}^{\min(t, H-1)} w_i}$$

where  $\mathbf{A}_{t-i}[i]$  denotes the action planned for time  $t$  within the chunk generated  $i$  steps prior, and  $w_i = \exp(-m \cdot i)$  (with  $m \geq 0$ ) is an exponential decay weight prioritizing recent predictions.

This smoothing eliminates step discontinuities in velocities and accelerations ( $\dot{\mathbf{q}}, \ddot{\mathbf{q}}$ ), driving joint controllers smoothly at 50 Hz without high-frequency chatter.

### End-to-End Numerical Example: Dynamic Grasping via Flow Matching and Action Chunking

To trace the continuous pipeline, consider a 7-DoF arm intercepting an object moving along a conveyor belt at 50 Hz.

1. **Conditioning Extraction** ( $t = 0$  s): The multimodal VLM processes the latest camera frame and the command *"Grasp the object on the conveyor"*. Concatenating  $[\mathbf{V}_{\text{llm}} \parallel \mathbf{T}_{\text{emb}}]$ , the backbone compresses semantic context into:

$$\mathbf{c} = f_{\text{VLA}}(\mathbf{S}_{\text{input}}) \in \mathbb{R}^{4096}$$

This vector guides the Action Head toward the operational region where interception should occur.

2. **Trajectory Chunk Initialization (Pure Noise)**: Assume a planning horizon of  $H = 4$  steps at 50 Hz ( $\Delta t = 0.02$  s, covering an 80 ms interval). The action at step  $h$  consists of Cartesian deltas and gripper state:  $\mathbf{u}_h = [\Delta x, \Delta y, \Delta z, \Delta \phi, \Delta \theta, \Delta \psi, \text{grip}] \in \mathbb{R}^7$ .

At flow step  $s = 0$ , the Action Head draws standard Gaussian noise:

$$\mathbf{A}^{(0)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \in \mathbb{R}^{4 \times 7}$$

Focusing for clarity on the forward Cartesian translation  $X$  across the 4 steps:

$$\mathbf{A}_{[:, \Delta x]}^{(0)} = \begin{bmatrix} +0.82 \\ -1.15 \\ +0.34 \\ -0.61 \end{bmatrix} \quad (\text{random dimensionless values})$$

3. **ODE Integration via Euler's Method** ( $N_{\text{steps}} = 2$ ,  $\Delta s = 0.5$ ): The Action Head parametrizes velocity field  $\mathbf{v}_\theta(\mathbf{A}_s, s, \mathbf{c})$ . Let the ground-truth target trajectory (in meters) be:

$$\mathbf{A}_{[:, \Delta x]}^* = \begin{bmatrix} +0.010 \\ +0.021 \\ +0.033 \\ +0.046 \end{bmatrix} \text{ m} \quad (\text{smooth forward progression along } X)$$

- **Step 1** ( $s = 0.0$ ): The network computes the initial velocity field conditioned on  $\mathbf{c}$  (via the Action Head neural network):

$$\mathbf{V}_{[:, \Delta x]}^{(0)} = \mathbf{v}_\theta(\mathbf{A}^{(0)}, 0.0, \mathbf{c}) \approx \mathbf{A}_{[:, \Delta x]}^* - \mathbf{A}_{[:, \Delta x]}^{(0)} = \begin{bmatrix} +0.010 - 0.82 \\ +0.021 - (-1.15) \\ +0.033 - 0.34 \\ +0.046 - (-0.61) \end{bmatrix} = \begin{bmatrix} -0.810 \\ +1.171 \\ -0.307 \\ +0.656 \end{bmatrix}$$

Euler state update ( $\Delta s = 0.5$ ):

$$\mathbf{A}_{[:, \Delta x]}^{(1)} = \mathbf{A}_{[:, \Delta x]}^{(0)} + 0.5 \cdot \mathbf{V}_{[:, \Delta x]}^{(0)} = \begin{bmatrix} 0.82 + 0.5(-0.810) \\ -1.15 + 0.5(1.171) \\ 0.34 + 0.5(-0.307) \\ -0.61 + 0.5(0.656) \end{bmatrix} = \begin{bmatrix} +0.415 \\ -0.564 \\ +0.186 \\ -0.282 \end{bmatrix}$$

- **Step 2** ( $s = 0.5$ ): The network evaluates velocity at midpoint:

$$\mathbf{V}_{[:, \Delta x]}^{(1)} = \mathbf{v}_\theta(\mathbf{A}^{(1)}, 0.5, \mathbf{c}) \approx \frac{\mathbf{A}_{[:, \Delta x]}^* - \mathbf{A}_{[:, \Delta x]}^{(1)}}{1.0 - 0.5} = \begin{bmatrix} -0.810 \\ +1.170 \\ -0.306 \\ +0.656 \end{bmatrix}$$

Final integration to  $s = 1.0$ :

$$\mathbf{A}_{[:, \Delta x]}^{(2)} = \mathbf{A}_{[:, \Delta x]}^{(1)} + 0.5 \cdot \mathbf{V}_{[:, \Delta x]}^{(1)} = \begin{bmatrix} +0.010 \\ +0.021 \\ +0.033 \\ +0.046 \end{bmatrix} \text{ m}$$

In two integration steps, initial noise converges to the continuous trajectory chunk  $\mathbf{A}^* \in \mathbb{R}^{4 \times 7}$ .

4. **Runtime Temporal Ensembling ( $t = 0.04$  s):** Because trajectories are replanned at each interval, at time  $t = 2\Delta t = 0.04$  s, the controller generates command  $\bar{\mathbf{u}}_{t=2}$ .

This instant is covered by two overlapping plans:

- Chunk generated at  $t = 0$ , where relative index for  $t = 2$  is  $\mathbf{u}_{t=0}[2] = +0.033$  m.
- New chunk replanned at  $t = 1$  with updated visual feedback, where relative index for that same instant is  $\mathbf{u}_{t=1}[1] = +0.031$  m.

Applying exponential weighting with decay factor  $m = 0.5$ :

$$w_0 = e^{-0.5 \cdot 0} = 1.0 \quad (\text{latest prediction, } t = 1), \quad w_1 = e^{-0.5 \cdot 1} \approx 0.606 \quad (\text{prior prediction, } t = 0)$$

The actual displacement sent to the joint controller is:

$$\bar{u}_{\Delta x}(t = 2) = \frac{1.0 \cdot (+0.031) + 0.606 \cdot (+0.033)}{1.0 + 0.606} = \frac{0.031 + 0.01999}{1.606} \approx +0.0317 \text{ m} \quad (+31.7 \text{ mm})$$

The transition between successive kinematic plans is smooth, maintaining continuity in acceleration and preventing joint vibrations.

### 3.2.4 3. Mathematical Formulation: Conditional Flow Matching (CFM)

Rather than relying on iterative stochastic denoising steps (as in traditional DDPM), CFM constructs a deterministic continuous probability flow mapping a known base distribution to the distribution of target trajectories.

**Interpolation Path Definition:** Let  $q(\mathbf{A})$  be the empirical distribution of demonstration trajectories and  $p_0(\mathbf{A}_0) = \mathcal{N}(\mathbf{0}, \mathbf{I})$  be a standard normal base distribution in  $\mathbb{R}^{H \times D}$ . Given demonstration trajectory  $\mathbf{A}_1 \sim q(\mathbf{A})$  and noise sample  $\mathbf{A}_0 \sim p_0(\mathbf{A}_0)$ , a linear interpolation path is defined over continuous variable  $s \in [0, 1]$  (*flow time*):

$$\mathbf{A}_s = (1 - s)\mathbf{A}_0 + s\mathbf{A}_1$$

Differentiating with respect to flow time  $s$ , the marginal vector velocity along the path is constant:

$$\frac{d\mathbf{A}_s}{ds} = \mathbf{A}_1 - \mathbf{A}_0$$

The vector  $(\mathbf{A}_1 - \mathbf{A}_0) \in \mathbb{R}^{H \times D}$  defines the straight-line trajectory transforming noise sample  $\mathbf{A}_0$  into target demonstration  $\mathbf{A}_1$ .

**Training Objective (CFM Loss):** Parametric network  $\mathbf{v}_\theta(\mathbf{A}_s, s, \mathbf{c})$  is trained to regress this velocity field by minimizing the mean squared error:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}(0,1), \mathbf{A}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \mathbf{A}_1 \sim q(\mathbf{A})} \|\mathbf{v}_\theta((1 - s)\mathbf{A}_0 + s\mathbf{A}_1, s, \mathbf{c}) - (\mathbf{A}_1 - \mathbf{A}_0)\|_F^2$$

where  $\|\cdot\|_F$  denotes the Frobenius norm on tensor  $H \times D$ . Training is highly efficient: no iterative diffusion loops are required during optimization; the loss evaluates random points  $s$  in a single forward pass.

### 3.2.5 4. Runtime Inference and ODE Numerical Integration

At runtime, the robot must extract trajectory  $\mathbf{A}_1$  without access to ground truth. This corresponds to numerically solving an Initial Value Problem (IVP) over  $s \in [0, 1]$ :

$$\begin{cases} \frac{d\mathbf{A}_s}{ds} = \mathbf{v}_\theta(\mathbf{A}_s, s, \mathbf{c}) \\ \mathbf{A}_{s=0} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \end{cases}$$

Because Flow Matching encourages straight probability paths (*straight flows*), integration converges in very few discretization steps  $N_{\text{steps}} \in [4, 8]$  using explicit solvers like **Forward Euler**:

---

**Algorithm 1** Trajectory Synthesis via Flow Matching ODE Integration

---

```
1: Input: Conditioning  $\mathbf{c} = f_{\text{VLA}}(\mathbf{S}_{\text{input}})$ , step count  $N$ , step size  $\Delta s = \frac{1}{N}$ 
2: Initial Sampling:  $\mathbf{A}^{(0)} \sim \mathcal{N}(\mathbf{0}_{H \times D}, \mathbf{I}_{H \times D})$ 
3: for  $k = 0$  to  $N - 1$  do
4:    $s_k \leftarrow k \cdot \Delta s$ 
5:    $\mathbf{V}^{(k)} \leftarrow \mathbf{v}_\theta(\mathbf{A}^{(k)}, s_k, \mathbf{c})$  ▷ Action Head forward pass
6:    $\mathbf{A}^{(k+1)} \leftarrow \mathbf{A}^{(k)} + \Delta s \cdot \mathbf{V}^{(k)}$  ▷ Euler integration step
7: end for
8: Output: Denoised action matrix  $\mathbf{A}^* = \mathbf{A}^{(N)} \in \mathbb{R}^{H \times D}$ 
```

---

ATTENTION: we discuss derivatives here to explain how the action head denoises data; concretely, the action head is a neural network trained with the derivative as its supervision target (learning to predict updates without running analytic differentiation at test time). The derivative target indicates the vector direction noise must follow to denoise; we cannot use a fixed static formula because the clean final trajectory is unknown at runtime.

### ACTION HEAD ARCHITECTURE:

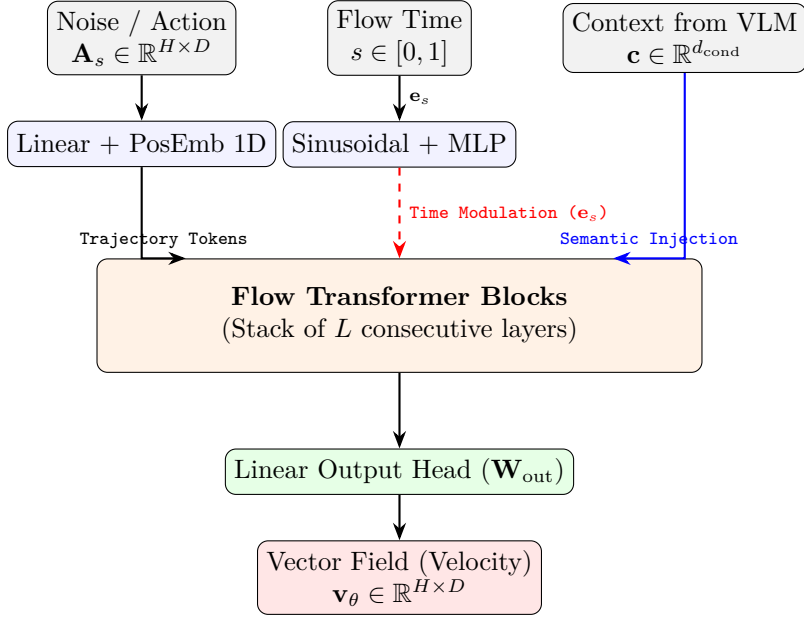


Figure 9: Macro-architecture of the Action Head. Kinematics, time, and semantic features are pre-processed and fed to the central Flow Transformer backbone.

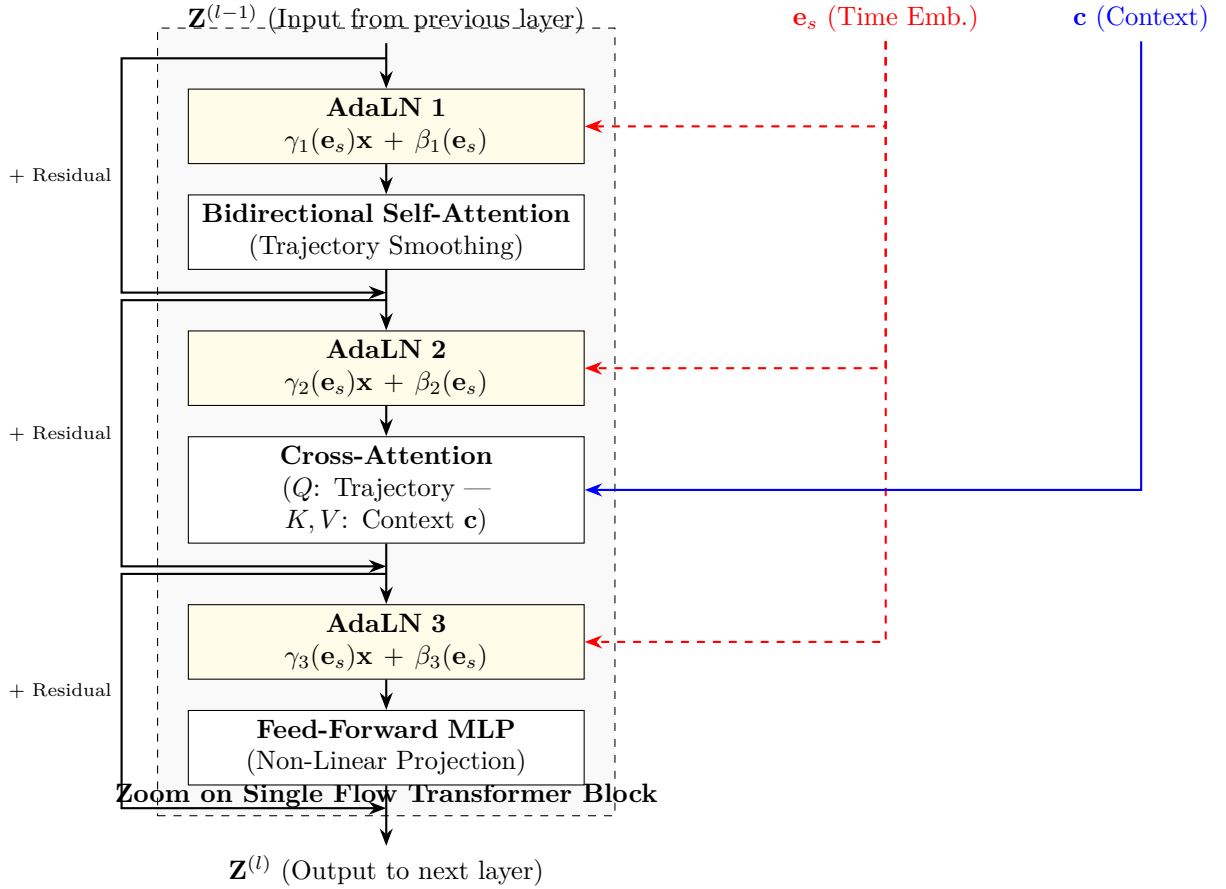


Figure 10: Micro-architecture details of an individual Transformer block. Time embedding  $s$  scales and shifts activations via *AdaLN*, whereas visual context  $\mathbf{c}$  enters solely through *Cross-Attention*.

### 3.3 On-Device Execution (Inference Time)

Once continuous trajectory  $\mathbf{A}_{t:t+k}$  is synthesized, it is rarely executed fully in an open-loop fashion. Robust robotic setups use **Temporal Ensembling**: at each control step  $t$ , the VLA receives a new camera frame and replans an overlapping future chunk. Actions are blended across overlapping horizons, filtering perceptual noise and accommodating physical manipulator dynamics on arms such as Franka or Dobot models.

## 4 Nature of the Visual Representation: Static Sensing vs. Dynamic Actuation

A core consideration in Vision-Language-Action architectures is the discrepancy between real-world environmental dynamics and model inference: **most modern VLAs process static visual snapshots (individual frames or small observation windows), bypassing the computational cost of continuous long-term video processing.**

### 4.1 The Quasi-Markovian Problem Formulation

In standard imitation learning (Behavior Cloning), closed-loop control is framed under an explicit or weak Markovian assumption. The conditional distribution over future actions is:

$$\pi_{\theta}(\mathbf{A}_{t:t+H} \mid \mathbf{o}_t, \mathbf{l}) \approx \pi_{\theta}(\mathbf{A}_{t:t+H} \mid \mathbf{I}_t, \mathbf{q}_t, \mathbf{l})$$

where  $\mathbf{I}_t$  is the single frame at time  $t$ ,  $\mathbf{q}_t$  denotes instant joint proprioception, and  $\mathbf{l}$  represents the task instruction.

#### 4.1.1 The Training Paradox:

During supervised training on teleoperation demonstrations, the underlying data represents continuous, high-frequency video streams alongside synchronous robot telemetry. However, data loading samples these streams independently:

- The training pipeline extracts isolated tuples  $(\mathbf{I}_t, \mathbf{l}, \mathbf{A}_{t:t+H}^*)$ , treating them as independent and identically distributed samples (*i.i.d.* batch sampling).
- The visual backbone (often a Vision Transformer such as SigLIP, CLIP, or DinoV2) processes static 2D spatial patches  $\mathbb{R}^{C \times W \times H}$ , lacking 3D temporal attention layers or convolutions along time.
- Task understanding ("*what the robot must achieve*") does not emerge from tracking visual history across video frames, but from statistical grounding between language command  $\mathbf{l}$ , static geometry in  $\mathbf{I}_t$ , and demonstrated future motions  $\mathbf{A}_{t:t+H}^*$ .

**ATTENTION:** during training, every frame contributes to a prediction of future actions (the chunk), and loss is computed against executed actions; weights are updated in parallel across the chunk via partial derivatives. During inference, the system replans at roughly 10–15 times per second, meaning not all incoming camera frames trigger a forward pass.

### 4.2 Physical Constraints of Single-Frame Perception

Decoupling static visual perception from dynamic action creates trade-offs impacting real-world execution:

1. **Differential Ambiguity (Velocity Invariance):** A single image  $\mathbf{I}_t$  records instantaneous spatial geometry, lacking scene velocity information ( $\dot{\mathbf{x}}_{\text{target}}$ ). In dynamic scenarios (e.g., conveyor belts, human handovers), a single frame cannot distinguish whether an object is stationary, approaching, or receding, introducing tracking errors in intercept trajectories.
2. **Vulnerability to Dynamic Occlusions:** If the robot arm briefly occludes the target or gripper from the view of an external camera, the policy exhibits *episodic amnesia*: lacking recurrent hidden states to track occluded objects, trajectory synthesis can degrade due to missing visual features.

### 4.3 Architectural Compensation Strategies

To address these limitations without incurring the high inference latency of native video architectures (such as Video Foundation Models operating on hundreds of frames), practical systems implement three complementary solutions:

#### 4.3.1 a) Multi-Frame Sliding Window (Bounded Temporal Stacking):

The Vision Encoder receives a short sequence of recent frames sampled at fixed intervals, e.g.,  $\mathbf{S}_{\text{vis}} = [\mathbf{I}_{t-2\Delta t}, \mathbf{I}_{t-\Delta t}, \mathbf{I}_t]$ . Processing this compact set of frames enables the attention layers to capture implicit optical flow, recovering object velocities with low computational overhead.

#### 4.3.2 b) Historical Proprioception Conditioning:

While camera frame acquisition and processing runs at moderate rates (10–30 Hz), motor encoders operate reliably above 500 Hz. Providing the network with recent joint positions and velocities  $\mathbf{Q}_{\text{hist}} = [\mathbf{q}_{t-k}, \dots, \mathbf{q}_t]$  provides direct observability of the robot’s own velocity and acceleration states.

#### 4.3.3 c) Shifting Dynamics into Future Horizons (Action Chunking):

Because vision is processed primarily as static snapshots at moderate rates, dynamic continuity is handled by the future prediction horizon. The model outputs  $H$  coordinated, non-causally constrained kinematic setpoints simultaneously, as detailed in preceding sections.